```
cy.server()
cy.route("/tickets", {id: 1, concert_id: 3, row: 2, seat: 3})
```

In this case, the first argument to route is a string or regular expression. Or it's two arguments where the first one is an HTTP method, and the second is the string or regex. The last argument is a string, array, object, or function.

Once a route is set, then HTTP requests that match the string or regex argument are not routed to the server but instead return the last argument. Requests that don't match a Cypress-defined route just go to the underlying server as expected.

This gets a little more interesting in conjunction with the cy.fixture() command. Cypress lets you keep fixture data, typically JSON or a string in files, in the cypress/fixtures directory. You can then use cy.fixture() to access the file and then use the data as the result of a route.

So if we've got a file cypress/fixtures/tickets.json, we could then do this:

```
cy.server()
cy.fixture("tickets.json").as("tickets")
cy.route('POST', '**/tickets', '@tickets')
```

Then any post request to tickets in your page will return the fixture file, making that data available for testing.

Troubleshooting

Cypress has a few ways to let you see what's going on during testing. First off, each step in a test produces a snapshot that you can look at in the test runner and also inspect via the browser console that you are using. You can also trigger a screenshot to the cypress/screenshots directory by using the screenshot command.

You can send an arbitrary message to the browser console log with cy.log("message"), and you can have Cypress log the result of any chain of message by placing debug() at the end of the chain. In the next chapter, we go into the specifics of using the Cypress test runner to debug Cypress tests.